Systems Security - Assignment 2

Group 8
Darius Muresan **Jendrik Engels** **Vlad Vladutu**

March 27, 2026

Compile and usage constructions

Solution for Task 1 and Task 4 have been implemented in python (`network/Task_1_Jendrik`, `network/Task_4_Darius`. Therefore, it is sufficient to install the imported libraries from the beginning of the code and then run the files.

For task 1: run the client.py and the server.py.

For task 4 almost the same, run the tls_client.py and tls_server.py

Task 2 has been implemented in c. It has a Makefile for the build, and then the usage is simply:

`task_04_test.ipynb` contains a simple analysis of the RSA-OAEP implementation. It doesn't need to be ran, it can be viewed from github.

We also have `network/task1_vlad` and `network/task3_vlad` in C but they are not covered in this report. They have usefull READMEs for each.

1 Task 1 - Encrypted ICMP Covert Channel

1.1 Sending Packets

For this task, we implemented two classes: `IcmpPacketSender` and `IcmpPacketServer`. The `IcmpPacketSender` class can be initialized with the port to which the message should be sent, the TTL (Time to Live), and an ID. The class provides the methods `encrypt`, `calculate_checksum`, and `send_icmp_packet`. We create an object and retrieve the message and the receiver's IP address from the console input. We then call the `send_icmp_packet` method on the object. This method works as follows: we set the ICMP type to 47, as specified in the task description. We then construct the payload by encoding the message and encrypting it with the `encrypt` function. For encryption, we use AES in GCM mode with a preshared key. GCM also provides integrity and authentication via an authentication tag, making it a wiser choice over modes such as CBC, which provide confidentiality only. We then construct the ICMP header using the `struct` library, including the ICMP type and code, a placeholder value of 0 for the checksum, the ID, and the sequence number. The checksum is then calculated over the header and the encrypted payload so that the header can be updated with the correct value. To calculate the checksum, we first ensure that the length of the data is even. If not, we append `\x00` so that we can always form 2-byte pairs. We then loop through all byte pairs and sum them as 16-bit words. After summing, the total may exceed 16 bits, which would not fit in the checksum field. To handle this, we take whatever overflowed above bit 16 and add it back into the lower 16 bits. Finally, we flip all bits and return the result. The computed checksum replaces the placeholder in the header, and the final packet is formed by concatenating the header and the encrypted payload. We then create a raw socket using the `socket` library and send the packet to the target IP using `sock.sendto()`.

1.2 Receiving Packets

The `IcmpPacketServer` class provides the methods `decrypt` and `receive_icmp_packet`. We create an object of the class and call `receive_icmp_packet`. This method creates a raw socket using the `socket` library and listens for incoming packets using `sock.recvfrom(65535)`, where 65535 is the maximum possible size of an IP packet. Since the raw packet includes the IP header in the first 20 bytes, we read the ICMP data starting at index 20. The first byte of the ICMP header holds the ICMP type, and if it equals 47, we extract the encrypted payload starting at index 28 (skipping the 8-byte ICMP header) and pass it to the `decrypt` function. If decryption and authentication succeed, the plaintext message is printed to the terminal. If the authentication tag verification fails, a `ValueError` is raised and the packet is discarded.

1.3 Note

The code was developed and tested on Windows, with the shell running under Administrator privileges, as raw sockets require elevated permissions on that platform.

2 Task 2 - TCP Session Hijacking via RST/ACK Injection

2.1 Approach and Libraries

This task is implemented in C using the `pcap` and `libnet` libraries. `pcap` enables passive capture of live traffic, while `libnet` handles raw packet construction and injection. Neither library provides a mechanism to drop packets in transit, so the attack operates on a best-effort basis: we react to observed packets as quickly as possible and, where necessary, send multiple forged packets to overcome the inherent race condition against the legitimate sender.

The network interface is hardcoded to `wlp1s0`. The kernel conveniently translates the Wi-Fi frame header into a standard Ethernet header of predictable length; we implemented a helper function `find_ip_start_eth` that identifies the Ethernet frame type and computes the header length, yielding a reliable byte offset to the IP payload.

2.2 Packet Filtering and Field Extraction

Rather than relying on BPF filter strings, we inspect the raw bytes of each captured packet manually, cross-referencing the standard IP and TCP header layouts. The following filters are applied in order:

- **IPv4 only:** discard any packet whose IP version field is not 4.
- **TCP only:** discard any packet whose IP protocol field is not 6.
- **Direction match:** compare the source and destination IP addresses against the target connection supplied as command-line arguments; discard non-matching packets.
- **Pure ACK:** target only packets where `tcp[13] == 0x10`, i.e. the ACK flag is set and no other flags are present, as these represent steady data-transfer segments.

From matching packets we extract the `seq` and `ack` sequence numbers for use in our forged replies. To avoid reacting to our own injected packets, we maintain a simple state variable tracking the highest sequence number seen so far and skip packets that do not advance it; while not entirely robust, this heuristic works reliably in practice.

2.3 TCP Timestamp Handling

Initial experiments with forged RST/ACK packets failed silently: `tcpdump` and Wireshark confirmed the packets were sent, yet the server discarded them without closing the connection. Closer inspection revealed that the legitimate connection uses TCP timestamps (the `TSval`/`TSecr` option pair), and the server validates these timestamps before accepting any segment. Packets lacking timestamp options were silently ignored.

We therefore implemented a dedicated function `parse_tcp_timestamps` to extract the `TSval` and `TSecr` fields from the TCP options of each captured packet. Since `TSval` increases monotonically with time, we increment the captured value by 100 and leave `TSecr` unchanged as a reasonable estimate for the forged packet.

One additional subtlety arose during construction: encoding only the two 4-byte timestamp values without padding caused the option block to be misinterpreted. The TCP options field must occupy a multiple of 4 bytes, so we prepend two NOP bytes (`0x01 0x01`) before the timestamp option kind and length bytes; this aligns the block correctly and is accepted by the server.

2.4 Packet Construction and Injection

With the extracted fields and computed timestamps in hand, we construct three layers:

1. **TCP header:** flags are set to `RST|ACK` (or `ACK` only, depending on the chosen method). Because neither variant carries a payload, the `seq` and `ack` values are taken directly from the captured packet unchanged.
2. **IP header:** all fields are mirrored from the captured packet (source/destination addresses, TTL, etc.), with only the total length recomputed to reflect the forged payload size.
3. **Injection:** the assembled packet is handed to `libnet_write()` for transmission onto the wire.

2.5 RST vs. Duplicate ACK

Two termination strategies were implemented and compared:

- **RST injection:** a single forged `RST|ACK` packet proved surprisingly reliable at tearing down the connection immediately.
- **Duplicate ACK flooding:** sending three consecutive duplicate ACKs triggers the server's fast-retransmit logic and ultimately causes the connection to stall. The required count is configurable in the source code.

2.6 Testing

To obtain a long-lived TCP transfer suitable for demonstrating the attack, we simulated a large download:

```
curl -o /dev/null --limit-rate 100k \
    http://ipv4.download.thinkbroadband.com/100MB.zip
```

The server IP (`80.249.99.148`) was resolved in advance using `getent hosts`. The attack was verified by observing the transfer terminated abruptly in both the terminal running the download and also in Wireshark.

3 Task 4 - Mini TLS-based VPN tunneling

Compile and usage constructions for all solutions

All solutions for this task have been implemented in Python 3. The environment relies on Docker containers to simulate the network. No external Python libraries need to be installed, as the code relies entirely on built-in modules (`os`, `struct`, `fcntl`, `socket`, `ssl`, `select`).

To run the implementation and reproduce the encrypted tunnel, follow these steps using four separate terminal sessions attached to the Docker containers:

- Terminal 1 (Server): Access the `server-router` container and execute the server script: `python3 tls_server.py`. Leave it running to listen for incoming connections.

- Terminal 2 (Client): Access the `client-10.9.0.5` container and execute the client script: `python3 tls_client.py`. A message confirming the established TLS connection will appear.

- Terminal 3 (Traffic Capture): Open a new session in the `client-10.9.0.5` container and start `tcpdump` to record traffic on the external interface: `tcpdump -i eth0 -w /volumes/encrypted_traffic.pcap`.

- Terminal 4 (Triggering Traffic): Open a final session in the `client-10.9.0.5` container and generate traffic intended for the protected network: `ping -c 4 192.168.60.5`. Once the ping finishes, stop the capture in Terminal 3 and analyze the `.pcap` file in Wireshark.

3.1 Public Key Infrastructure (PKI) and Mutual Authentication

For a TLS connection a trust framework must be defined to authenticate communicating parties. For this task, mutual authentication is required, meaning both the client and the server must verify each other's identities. To achieve this, we first established our own Certificate Authority. Using `openssl`, we generated a self-signed root certificate (`ca.crt`) and its private key. At the same time, we generated CSRs for both the VPN Server and the VPN Client. These requests were then signed using the CA's private key, resulting in `server.crt` and `client.crt`. By deploying the `ca.crt` on both endpoints, both the client and server can mathematically verify that the other party's certificate was issued by our trusted authority.

3.2 VPN Server Implementation

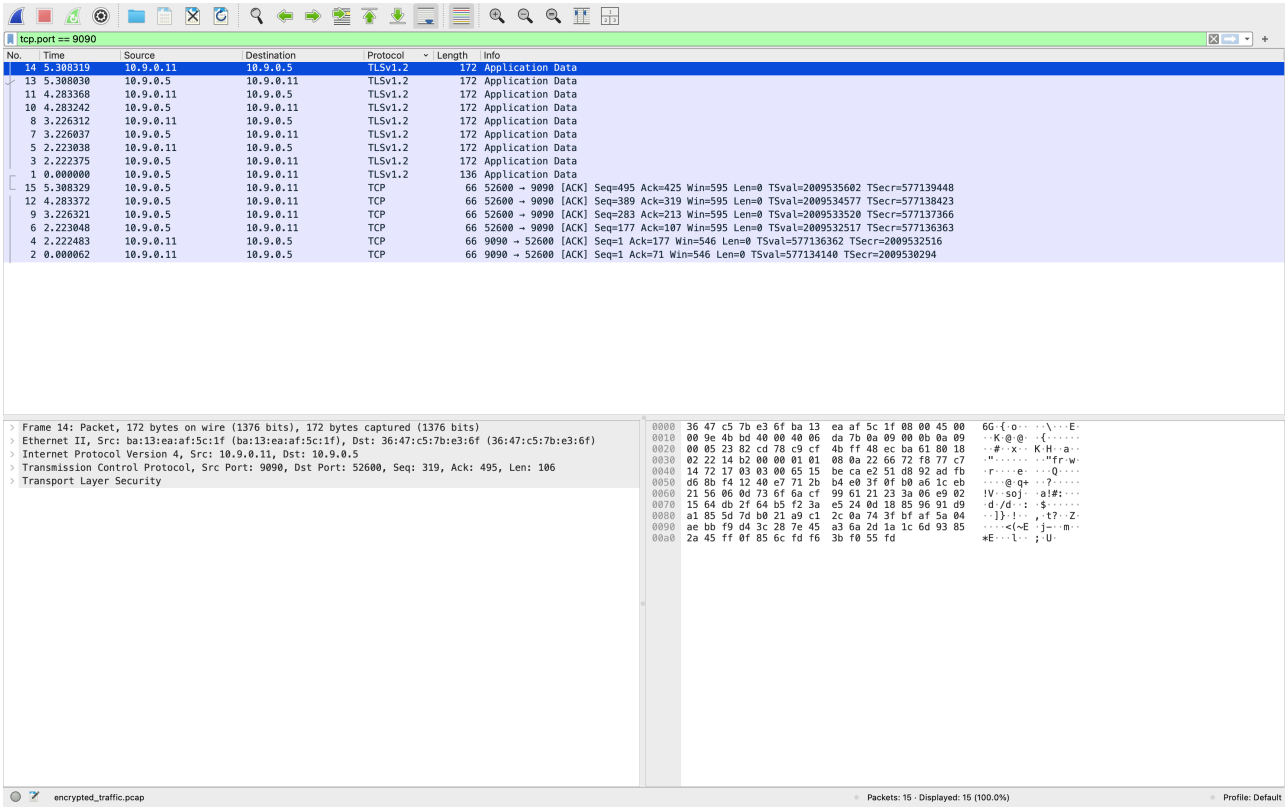
First, we created a TUN interface to receive raw IP packets directly from user-space programs. We interact with the `/dev/net/tun` device using the `fcntl` and `struct` libraries to allocate a virtual interface (e.g., `tun0`). Next, we use the Python ssl module to create an `SSLContext` configured for a server. We load the server's certificate chain (`server.crt` and `server.key`) and explicitly set `context.verify_mode = ssl.CERT_REQUIRED`. This enforces mutual authentication by demanding a valid certificate from any connecting client, verified against our loaded `ca.crt`. A standard TCP socket is then created and bound to port 9090. When the client connects, the connection is wrapped using `context.wrap_socket()`, upgrading the standard TCP socket into a secure TLS tunnel. Finally, we implemented a routing loop using the select module. If data is available on the TUN interface, it is read and sent into the TLS socket (`ssock.sendall()`), encrypting it automatically. If data arrives on the TLS socket, it is decrypted automatically by `recv()` and written directly into the TUN interface, injecting it into the internal network. We assigned the `IP 10.4.2.99` to the server's TUN interface and bring it up using `os.system()`.

3.3 VPN Client Implementation

The client implementation (`tls_client.py`) acts as the entry point to the tunnel. For the security context, it is configured as `PROTOCOL_TLS_CLIENT`. It loads the `ca.crt` to verify the server's identity and its own `client.crt/client.key` pair to prove its identity to the server. Since we are connecting directly via an IP address (`10.9.0.11`) rather than a domain name, we disable hostname verification (`check_hostname = False`) to prevent SSL handshake errors. To ensure traffic reaches the tunnel, we assigned the IP `10.4.2.5` to the client's TUN interface and, added a routing rule: `ip route add 192.168.60.0/24 dev tun0`. This instructs the operating system to direct any packets destined for the target subnet.

3.4 Verification and Traffic Analysis

To prove the confidentiality of the tunnel, we generated ICMP Echo Request packets using the `ping` command aimed at a host inside the protected network (`192.168.60.5`). By using `tcpdump` to monitor the external-facing interface (`eth0`) of the client container, we captured the packets traversing the simulated internet. Analyzing the resulting capture in Wireshark filtered by `tcp.port == 9090` shows exclusively TCP and TLSv1.2/TLSv1.3 protocols. The packet payloads are labeled as Application Data, confirming that the ICMP packets are fully encapsulated and encrypted. No plaintext routing information regarding the internal `192.168.60.x` network is visible to outside observers.



References